CSE464 Advanced Database Management Systems

Provenance-Enabled Relational Database for E-Commerce

A Project Report Submitted in Partial Fulfilment of the Requirements for the Course CSE464

Submitted By-

Name	Student ID
Md. Saleh Rafi	2022-3-60-141
Zahin Shariar	2023-3-60-213
Lamia Bari	2023-3-60-230

Submitted To-

Antu Chowdhury

Lecturer

Department of Computer Science & Engineering

East West University

Table of Contents

1. Introduction

In modern data-intensive applications, understanding the origin and transformation of data is essential for transparency, auditing, debugging, and regulatory compliance. This project implements a Provenance-Enabled Relational Database Management System (RDBMS) for a simulated e-commerce scenario.

The system captures why, where, and how data evolves over time through three core mechanisms: a normalized relational schema, dedicated audit tables with full change history, and SQL triggers that automatically record every data modification without requiring changes to application code.

1.1 Project Objectives

a. To design a normalized relational database schema for the selected e-commerce application.
b. To identify critical entities where data provenance plays a significant role.
c. To develop audit mechanisms for capturing provenance information during DML operations (INSERT, UPDATE, DELETE).
d. To construct and execute SQL queries for retrieving and analysing provenance data.
e. To optionally develop a graphical user interface (GUI) for demonstrating provenance tracking.
f. To analyse and explain different types of provenance, including Why, Where, and How provenance.

1.2 Technology Stack

Component	**Technology**	**Purpose**
Database (Production)	Oracle SQL	Primary RDBMS with full trigger support
Database (Development)	SQLite 3	Local development, no installation needed
Backend	Python 3 + Flask	Lightweight web server & REST API
Frontend	HTML5 / CSS3 / JS	Single-page browser interface
Schema Migration	SQL Scripts	Portable DDL for both Oracle and SQLite

1.3 Identifying Critical Entities for Provenance

A key objective of this project is to identify which entities in the e-commerce domain are most valuable to track for provenance. Not every table requires audit logging - the selection is driven by which data changes carry business, legal, or debugging significance.

Entity	**Why Provenance is Critical**	**Provenance Type(s)**
products	Price and availability changes directly affect customer trust and financial reporting. If a customer disputes a charge, the historical price at order time must be provable.	Where (price history), How (activation/deactivation trail)
orders	Order status is the primary lifecycle indicator. Knowing when and by whom an order moved from CREATED to PAID to SHIPPED is essential for customer support, dispute resolution, and compliance auditing.	How (status evolution), Where (actor accountability)

order_items	Quantity or price changes after an order is placed could indicate fraud or data entry errors. Provenance here explains why the final total is what it is.	Why (total composition), Why (quantity justification)
payments	Payment status transitions (CAPTURED -> VOID -> REFUNDED) have direct financial consequences. Every change must be attributable to a specific user and timestamp for accounting purposes.	How (payment lifecycle), Where (refund trail)
customers	Email or name changes must be auditable for GDPR compliance and to prevent account hijacking from going undetected.	Where (contact info history)

These five entities collectively cover the full transaction lifecycle of an e-commerce system. Together they enable complete end-to-end provenance: from product listing, through order placement and item selection, to payment capture - every critical state change is recorded

2. Database Design

The database schema models a realistic e-commerce application with five core entities. The schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and enforce referential integrity.

2.1 Entity-Relationship Overview

The five core entities and their relationships are: **Customers** place **Orders**, which contain one or more **Order Items** (referencing **Products**). Each Order may have one or more **Payments** associated with it.

2.2 Table Definitions

customers

Column	Type	Constraints	Description
customer_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique customer identifier
name	VARCHAR(100)	NOT NULL	Customer full name
email	VARCHAR(150)	UNIQUE, NOT NULL	Contact email address
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

products

Column	Type	Constraints	Description
product_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique product identifier
name	VARCHAR(120)	NOT NULL	Product display name
price	NUMERIC(10,2)	NOT NULL, CHECK >= 0	Current selling price

active_flag	CHAR(1)	DEFAULT 'Y', CHECK IN ('Y','N')	Whether product is listed
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

orders

Column	Type	Constraints	Description
order_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique order identifier
customer_id	INTEGER	FK -> customers	Order owner
order_date	DATE	DEFAULT CURRENT_DATE	Date order was placed
status	VARCHAR(20)	CHECK IN (CREATED,PAID,SHIPPED,CANCELLED,REFUNDED)	Lifecycle stage
total_amount	NUMERIC(12,2)	DEFAULT 0, CHECK >= 0	Auto-computed order total

order_items

Column	Type	Constraints	Description
order_id	INTEGER	PK (composite), FK -> orders	Parent order
product_id	INTEGER	PK (composite), FK -> products	Product being ordered
quantity	NUMERIC(10,2)	NOT NULL, CHECK > 0	Quantity ordered
unit_price	NUMERIC(10,2)	NOT NULL, CHECK >= 0	Price at time of order
line_total	NUMERIC(12,2)	GENERATED (qty × price)	Computed line value

payments

Column	Type	Constraints	Description
payment_id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique payment identifier
order_id	INTEGER	FK -> orders	Associated order
amount	NUMERIC(12,2)	NOT NULL, CHECK > 0	Payment amount
payment_date	DATE	DEFAULT CURRENT_DATE	Date of payment
method	VARCHAR(20)	CHECK IN (CARD,CASH,BKASH,NAGAD,BANK)	Payment channel
status	VARCHAR(20)	DEFAULT 'CAPTURED'	Payment lifecycle status

3. Audit Table Design

To capture data provenance, a dedicated audit table is paired with each core table. Audit tables are append-only: records are only ever inserted, never updated or deleted, preserving an immutable history of all changes.

3.1 Audit Table Structure

Every audit table shares a common set of provenance columns in addition to entity-specific old/new value columns:

Column	Present In	Description
audit_id	All audit tables	Auto-incremented surrogate primary key
<entity>_id	All audit tables	Foreign reference to the affected row
operation	All audit tables	Type of change: INSERT, UPDATE, or DELETE
old_<field>	All audit tables	Value of the field before the change (NULL for INSERT)
new_<field>	All audit tables	Value of the field after the change (NULL for DELETE)
actor	All audit tables	Database user who performed the operation
operation_time	All audit tables	Exact timestamp of the change

3.2 Audit Tables Summary

Audit Table	Tracks	Captured Fields
audit_customers	customers	name, email
audit_products	products	name, price, active_flag
audit_orders	orders	status, total_amount
audit_order_items	order_items	quantity, unit_price
audit_payments	payments	amount, status

3.3 Sequence / Auto-ID Strategy

Oracle uses named sequences with different starting offsets per audit table to make audit IDs visually distinguishable in queries:

Sequence	Starts At	Used By
seq_audit_customers	1	audit_customers
seq_audit_products	10001	audit_products
seq_audit_orders	20001	audit_orders
seq_audit_order_items	30001	audit_order_items
seq_audit_payments	40001	audit_payments

SQLite uses AUTOINCREMENT on each table's primary key, which achieves the same result without explicit sequences.

4. Trigger / Procedure Implementation

SQL triggers are used to automatically populate audit tables whenever data changes. This approach requires zero changes to application code - the database itself enforces provenance tracking.

4.1 Trigger Design Principles

- One trigger per table covers all three DML operations (INSERT, UPDATE, DELETE)
- Triggers fire AFTER the DML statement so the operation cannot be prevented by trigger logic
- FOR EACH ROW ensures every affected row gets its own audit record
- The actor field captures the current database session user automatically

4.2 Trigger List

Trigger Name	Table	Events	Side Effect
trg_audit_customers	customers	INSERT, UPDATE, DELETE	Writes to audit_customers
trg_audit_products	products	INSERT, UPDATE, DELETE	Writes to audit_products
trg_audit_orders	orders	INSERT, UPDATE, DELETE	Writes to audit_orders
trg_audit_order_items	order_items	INSERT, UPDATE, DELETE	Writes to audit_order_items + recomputes order total
trg_audit_payments	payments	INSERT, UPDATE, DELETE	Writes to audit_payments

4.3 Products Trigger (Oracle Example)

The following trigger captures all changes to the products table:

```
CREATE OR REPLACE TRIGGER trg_audit_products
AFTER INSERT OR UPDATE OR DELETE ON products
FOR EACH ROW
BEGIN
 IF INSERTING THEN
  INSERT INTO audit_products(audit_id,product_id,operation,
       new_name,new_price,new_active,actor)
  VALUES(seq_audit_products.NEXTVAL,:NEW.product_id,'INSERT',
       :NEW.name,:NEW.price,:NEW.active_flag,USER);
 ELSIF UPDATING THEN
  INSERT INTO audit_products(audit_id,product_id,operation,
       old_name,new_name,old_price,new_price,old_active,new_active,actor)
  VALUES(seq_audit_products.NEXTVAL,:OLD.product_id,'UPDATE',
       :OLD.name,:NEW.name,:OLD.price,:NEW.price,
       :OLD.active_flag,:NEW.active_flag,USER);
```

```
 ELSIF DELETING THEN
  INSERT INTO audit_products(audit_id,product_id,operation,
       old_name,old_price,old_active,actor)
  VALUES(seq_audit_products.NEXTVAL,:OLD.product_id,'DELETE',
       :OLD.name,:OLD.price,:OLD.active_flag,USER);
 END IF;
END;
/
```

:NEW and :OLD are Oracle pseudo-records giving access to the row values before and after the change. For INSERT, only :NEW is populated. For DELETE, only :OLD is populated. For UPDATE, both are available.

4.4 recompute_order_total Procedure

When order items are added, updated, or removed, the order's total_amount must be refreshed. This is handled by a dedicated stored procedure called from within the order_items trigger:

```
-- Oracle
CREATE OR REPLACE PROCEDURE recompute_order_total(p_order_id IN NUMBER) AS
BEGIN
 UPDATE orders SET total_amount = (
  SELECT NVL(SUM(quantity * unit_price), 0)
  FROM order_items WHERE order_id = p_order_id
 ) WHERE order_id = p_order_id;
END;
/
```

```
-- SQLite (inline in trigger)
UPDATE orders SET total_amount = (
 SELECT COALESCE(SUM(quantity*unit_price),0)
 FROM order_items WHERE order_id = NEW.order_id
) WHERE order_id = NEW.order_id;
```

4.5 Order Items Trigger (Oracle)

This is the most complex trigger. After recording the audit entry, it calls recompute_order_total to keep the parent order's total_amount in sync:

```
CREATE OR REPLACE TRIGGER trg_audit_order_items
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW
BEGIN
 IF INSERTING THEN
  INSERT INTO audit_order_items(audit_id,order_id,product_id,
       operation,new_quantity,new_price,actor)
  VALUES(seq_audit_order_items.NEXTVAL,:NEW.order_id,:NEW.product_id,
       'INSERT',:NEW.quantity,:NEW.unit_price,USER);
  recompute_order_total(:NEW.order_id);
 ELSIF UPDATING THEN
```

```
  INSERT INTO audit_order_items(audit_id,order_id,product_id,operation,
      old_quantity,new_quantity,old_price,new_price,actor)
  VALUES(seq_audit_order_items.NEXTVAL,:OLD.order_id,:OLD.product_id,
      'UPDATE',:OLD.quantity,:NEW.quantity,:OLD.unit_price,:NEW.unit_price,USER);
  recompute_order_total(:NEW.order_id);
 ELSIF DELETING THEN
  INSERT INTO audit_order_items(audit_id,order_id,product_id,
      operation,old_quantity,old_price,actor)
  VALUES(seq_audit_order_items.NEXTVAL,:OLD.order_id,:OLD.product_id,
      'DELETE',:OLD.quantity,:OLD.unit_price,USER);
  recompute_order_total(:OLD.order_id);
 END IF;
END;
/
```

4.6 Payments Trigger (Oracle)

Records every payment lifecycle event including captures, voids, and refunds:

```
CREATE OR REPLACE TRIGGER trg_audit_payments
AFTER INSERT OR UPDATE OR DELETE ON payments
FOR EACH ROW
BEGIN
 IF INSERTING THEN
  INSERT INTO audit_payments(audit_id,payment_id,order_id,
      operation,new_status,new_amount,actor)
  VALUES(seq_audit_payments.NEXTVAL,:NEW.payment_id,:NEW.order_id,
      'INSERT',:NEW.status,:NEW.amount,USER);
 ELSIF UPDATING THEN
  INSERT INTO audit_payments(audit_id,payment_id,order_id,operation,
      old_status,new_status,old_amount,new_amount,actor)
  VALUES(seq_audit_payments.NEXTVAL,:OLD.payment_id,:OLD.order_id,'UPDATE',
      :OLD.status,:NEW.status,:OLD.amount,:NEW.amount,USER);
 ELSIF DELETING THEN
  INSERT INTO audit_payments(audit_id,payment_id,order_id,
      operation,old_status,old_amount,actor)
  VALUES(seq_audit_payments.NEXTVAL,:OLD.payment_id,:OLD.order_id,
      'DELETE',:OLD.status,:OLD.amount,USER);
 END IF;
END;
/
```

4.7 recompute_order_total Procedure

When order items are added, updated, or removed, the order's total_amount must be refreshed. This is handled by a dedicated stored procedure called from within the order_items trigger:

```
-- Oracle
CREATE OR REPLACE PROCEDURE recompute_order_total(p_order_id IN NUMBER) AS
```

```
BEGIN
  UPDATE orders SET total_amount = (
    SELECT NVL(SUM(quantity * unit_price), 0)
    FROM order_items WHERE order_id = p_order_id
  ) WHERE order_id = p_order_id;
END;
/
```

```
-- SQLite (inline in trigger)
UPDATE orders SET total_amount = (
  SELECT COALESCE(SUM(quantity*unit_price),0)
  FROM order_items WHERE order_id = NEW.order_id
) WHERE order_id = NEW.order_id;
```

5. Provenance Queries

Provenance queries allow users to interrogate the audit log to answer three fundamental questions about data history.

5.1 Types of Provenance

Type	Question Answered	Source Tables
Why-Provenance	Why does this data value exist? What contributed to it?	Core tables (order_items, products)
Where-Provenance	Where did this value come from? What was its history?	Audit tables (audit_products, audit_orders)
How-Provenance	How did this value come to be? What sequence of operations?	Audit tables (audit_orders, audit_order_items)

5.2 Query 1 - Why-Provenance: Order Total

Question: Why is Order 1's total 1549.99?

```
SELECT oi.product_id, p.name, oi.quantity, oi.unit_price,
     ROUND(oi.quantity * oi.unit_price, 2) AS line_total
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
WHERE oi.order_id = 1;
```

Interpretation: The total is the sum of all line totals. Laptop (1 × 1099.99) + Headphones (3 × 150.00) = 1549.99.

5.3 Query 2 - Where-Provenance: Laptop Price History

Question: How did the Laptop price evolve?

```
SELECT ap.operation_time, ap.operation, ap.old_price, ap.new_price, ap.actor
FROM audit_products ap
JOIN products p ON p.product_id = ap.product_id
WHERE p.name = 'Laptop'
ORDER BY ap.operation_time;
```

Interpretation: Shows the INSERT at 1000.00 and the subsequent UPDATE to 1099.99, with timestamps and actor.

5.4 Query 3 - How-Provenance: Order Status Lifecycle

Question: How did Order 1 move through its lifecycle?

```
SELECT operation_time, operation, old_status, new_status, actor
FROM audit_orders
WHERE order_id = 1
ORDER BY operation_time;
```

Interpretation: Reveals the full sequence CREATED -> PAID -> SHIPPED, each step with its timestamp and responsible actor.

5.5 Query 4 - Where-Provenance: Actor Activity

Question: What did the current user do on Orders?

```
SELECT actor, operation, order_id, operation_time, old_status, new_status
FROM audit_orders
WHERE actor = USER   -- Oracle syntax
-- WHERE actor = 'app_user'  -- SQLite syntax
ORDER BY operation_time;
```

5.6 Query 5 - Why-Provenance: Item Quantity Change

Question: Why does Order 1 have 3 Headphones instead of 2?

```
SELECT operation_time, operation, old_quantity, new_quantity, actor
FROM audit_order_items
WHERE order_id = 1 AND product_id = 3
ORDER BY operation_time;
```

Interpretation: Shows the INSERT at quantity 2, followed by an UPDATE to 3. This proves the quantity was intentionally changed post-insert.

6. GUI

A lightweight single-page web application was built using Python Flask to allow interactive exploration of all database tables and provenance queries without writing any SQL manually.

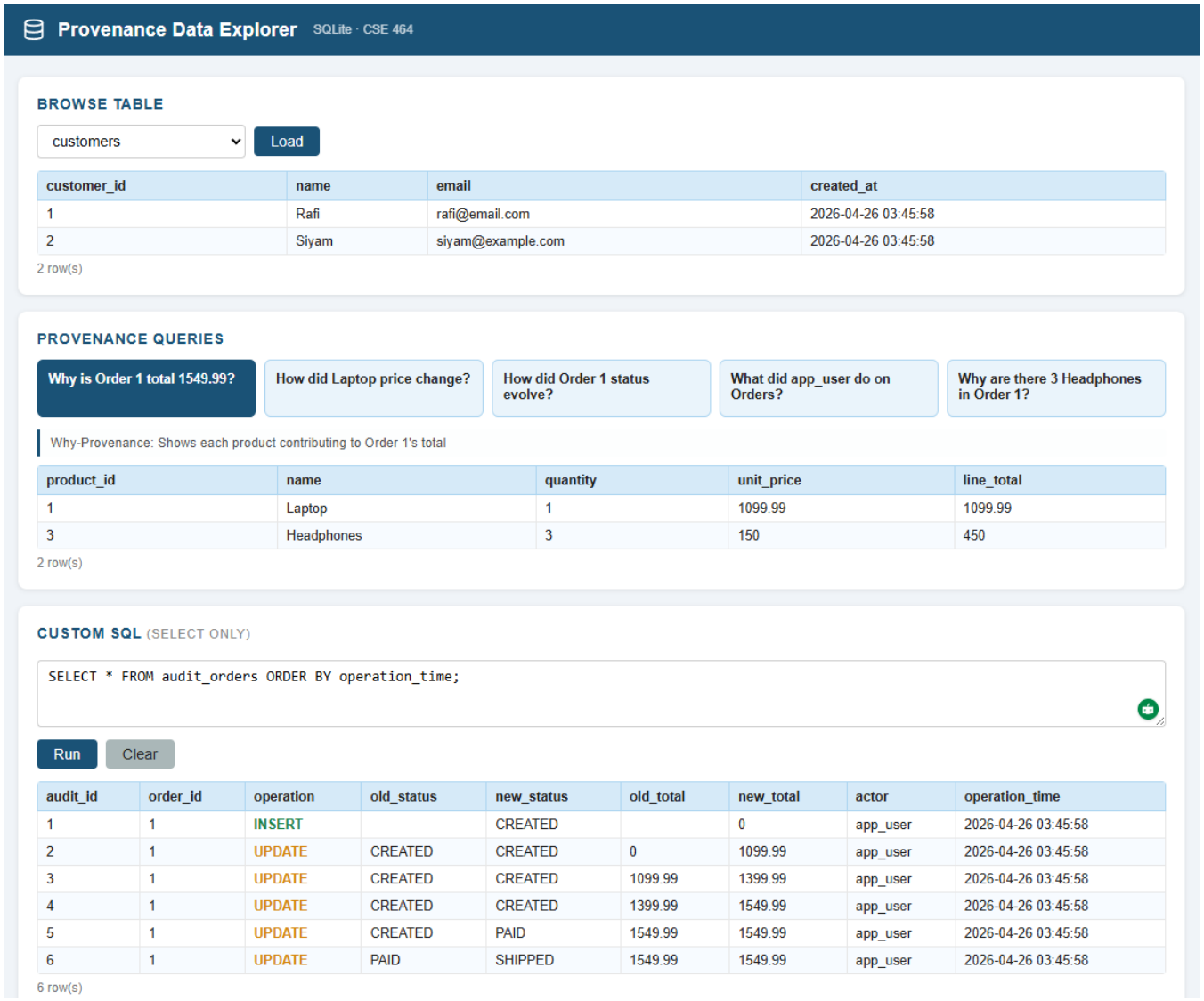

Fig-1: The single page GUI application.

6.1 Architecture

Layer	Technology	File
Web Server	Python Flask (WSGI)	app.py
Database	SQLite 3 (built-in Python)	provenance.db (auto-generated)
Schema & Data	Pure SQL DDL + DML	schema_sqlite.sql
Frontend	Inline HTML/CSS/JS	app.py (HTML string)
API	3 JSON REST endpoints	app.py

6.2 Features

- Browse Table: Select any of the 10 tables (5 core + 5 audit) and view up to 200 rows.
- Provenance Queries: 5 pre-built clickable provenance queries with descriptions.
- Custom SQL: Write and execute any SELECT statement with live results.
- Color-coded operations: INSERT rows appear green, UPDATE orange, DELETE red.
- No installation required beyond pip install flask.

6.3 Setup & Run Instructions

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)

Step 1 - Install dependency

```
pip install flask
```

Step 2 - File Structure

```
project/
  app.py
  schema_sqlite.sql
  project_report.pdf
  provenance.db
```

Step 3 - Run

```
python app.py
```

The first run automatically creates provenance.db and seeds it with sample data. A confirmation message is printed:

```
DB ready. Open http://localhost:5000
```

Step 4 - Open browser

```
http://localhost:5000
```

To reset the database and re-seed sample data, delete provenance.db and restart the app.

7. Oracle SQL vs SQLite - Differences

The project schema was originally designed for Oracle SQL and later ported to SQLite for local development. The following table documents the key syntax differences:

Feature	Oracle SQL	SQLite
Auto-increment PK	SEQUENCE + BEFORE INSERT trigger	INTEGER PRIMARY KEY AUTOINCREMENT
Current timestamp	SYSTIMESTAMP	CURRENT_TIMESTAMP
Current date	SYSDATE	CURRENT_DATE

Null-safe sum	NVL(SUM(...), 0)	COALESCE(SUM(...), 0)
String type	VARCHAR2(n)	TEXT
Number type	NUMBER(p,s)	REAL / INTEGER
Trigger syntax	IF INSERTING / ELSIF UPDATING	IF NEW / ELSIF (not supported; separate triggers)
Trigger per event	One trigger, IF INSERTING block	Separate trigger per event (INSERT/UPDATE/DELETE)
Virtual column	GENERATED ALWAYS AS (expr) VIRTUAL	Computed inline in queries (not stored)
Row limit	FETCH FIRST n ROWS ONLY	LIMIT n
Current user	USER	Hardcoded string 'app_user'
Block execution	DECLARE ... BEGIN ... END; /	Not applicable (no anonymous blocks)

8. Challenges and Lessons Learned

Mutating Table Errors

In Oracle, a row-level trigger on a table cannot query the same table being modified (ORA-04091 mutating table error). The recompute_order_total procedure was designed to work around this by updating the orders table from within the order_items trigger rather than querying order_items inside an orders trigger.

Trigger Granularity in SQLite

Unlike Oracle, SQLite does not support compound IF INSERTING / ELSIF UPDATING logic within a single trigger. Each DML event (INSERT, UPDATE, DELETE) requires its own separate trigger definition. This resulted in three triggers per table in SQLite versus one in Oracle.

Audit Table Redundancy vs. Completeness

Designing audit tables required careful balance between capturing enough information to answer all provenance queries and avoiding excessive storage overhead. The final design captures only the fields that are meaningful to track (status, price, quantity) rather than duplicating every column.

Consistency of Total Amount

Keeping orders.total_amount consistent with the sum of order_items.line_total required a stored procedure called from triggers. Without this, any quantity or price change would leave the order header out of sync with its detail lines.

9. Conclusion

This project successfully implemented a provenance-enabled RDBMS for an e-commerce application. The system demonstrates how audit tables and database triggers can be combined to capture a complete, immutable history of all data changes without modifying application-level code.

The three types of provenance - Why, Where, and How - were each demonstrated with concrete SQL queries against real audit data. The accompanying Flask web application provides an accessible GUI for exploring both raw table data and provenance query results interactively.

Future improvements could include extending provenance tracking to cover soft-delete patterns, adding IP address and application module logging to audit records, and building a visual timeline view of entity lifecycles in the GUI.

10. Appendix

A. File Structure

File	Description
schema_oracle.sql	Complete Oracle DDL: tables, sequences, triggers, sample data
schema_sqlite.sql	Complete SQLite DDL: tables, triggers, sample data (no sequences)
app.py	Flask web server + SQLite integration + HTML GUI (single file)
provenance.db	Auto-generated SQLite database file (created on first run)

B. Sample Data Summary

Entity	Records Seeded	Notes
customers	2	Rafi, Siyam
products	3	Laptop (price updated), Tablet (deactivated), Headphones
orders	1	Order 1 - status moved CREATED -> PAID -> SHIPPED
order_items	2	Laptop ×1, Headphones ×3 (quantity updated from 2)
payments	1	1549.99 via CARD, CAPTURED

----------EOF----------